

Universal and Fault-Tolerant Quantum Computing

Session 3 – Day 1

Dr. Shashank Gupta

SRM University-AP & Indian Institute of Technology Indore

3 July 2026



Learning Objectives

- Understand the definitions of exact vs. approximate universality.

Learning Objectives

- Understand the definitions of exact vs. approximate universality.
- Study the Solovay–Kitaev theorem and logical gate synthesis.

Learning Objectives

- Understand the definitions of exact vs. approximate universality.
- Study the Solovay–Kitaev theorem and logical gate synthesis.
- Learn the mathematical foundation of the Stabilizer Formalism.

Learning Objectives

- Understand the definitions of exact vs. approximate universality.
- Study the Solovay–Kitaev theorem and logical gate synthesis.
- Learn the mathematical foundation of the Stabilizer Formalism.
- Analyze standard QEC codes (bit-flip, phase-flip, Shor 9-qubit code).

Learning Objectives

- Understand the definitions of exact vs. approximate universality.
- Study the Solovay–Kitaev theorem and logical gate synthesis.
- Learn the mathematical foundation of the Stabilizer Formalism.
- Analyze standard QEC codes (bit-flip, phase-flip, Shor 9-qubit code).
- Explore Topological Quantum Codes (Kitaev's toric code & surface codes).

Learning Objectives

- Understand the definitions of exact vs. approximate universality.
- Study the Solovay–Kitaev theorem and logical gate synthesis.
- Learn the mathematical foundation of the Stabilizer Formalism.
- Analyze standard QEC codes (bit-flip, phase-flip, Shor 9-qubit code).
- Explore Topological Quantum Codes (Kitaev's toric code & surface codes).
- Grasp the Eastin–Knill theorem and magic state distillation.

Learning Objectives

- Understand the definitions of exact vs. approximate universality.
- Study the Solovay–Kitaev theorem and logical gate synthesis.
- Learn the mathematical foundation of the Stabilizer Formalism.
- Analyze standard QEC codes (bit-flip, phase-flip, Shor 9-qubit code).
- Explore Topological Quantum Codes (Kitaev's toric code & surface codes).
- Grasp the Eastin–Knill theorem and magic state distillation.
- Comprehend the threshold theorem and resource overhead scaling.

Why Fault Tolerance?

The Challenge of Noise

Physical qubits are highly susceptible to environmental decoherence, gate calibration errors, and readout noise.

Why Fault Tolerance?

The Challenge of Noise

Physical qubits are highly susceptible to environmental decoherence, gate calibration errors, and readout noise.

- Without error correction, noise propagates through gates and destroys logical coherence.

Why Fault Tolerance?

The Challenge of Noise

Physical qubits are highly susceptible to environmental decoherence, gate calibration errors, and readout noise.

- Without error correction, noise propagates through gates and destroys logical coherence.
- Large-scale quantum algorithms (e.g., Shor's algorithm requiring 10^9 gates) are unfeasible without active error suppression.

Why Fault Tolerance?

The Challenge of Noise

Physical qubits are highly susceptible to environmental decoherence, gate calibration errors, and readout noise.

- Without error correction, noise propagates through gates and destroys logical coherence.
- Large-scale quantum algorithms (e.g., Shor's algorithm requiring 10^9 gates) are unfeasible without active error suppression.
- Fault-tolerant thresholds ensure that if physical error rates are below a critical threshold ($\sim 1\%$), we can run computations of arbitrary length.

Universality – Exact vs. Approximate

Exact Universality: Any unitary $U \in SU(2^n)$ is represented *exactly* by a finite sequence of gates.

Universality – Exact vs. Approximate

Exact Universality: Any unitary $U \in SU(2^n)$ is represented *exactly* by a finite sequence of gates.

- Requires continuous parameter gate sets (e.g., $\{\text{CNOT}, R_z(\theta), R_y(\theta)\}$).

Universality – Exact vs. Approximate

Exact Universality: Any unitary $U \in SU(2^n)$ is represented *exactly* by a finite sequence of gates.

- Requires continuous parameter gate sets (e.g., $\{\text{CNOT}, R_z(\theta), R_y(\theta)\}$).
- Uncountably infinite group $SU(2^n)$ cannot be generated by a discrete set.

Universality – Exact vs. Approximate

Exact Universality: Any unitary $U \in SU(2^n)$ is represented *exactly* by a finite sequence of gates.

- Requires continuous parameter gate sets (e.g., $\{\text{CNOT}, R_z(\theta), R_y(\theta)\}$).
- Uncountably infinite group $SU(2^n)$ cannot be generated by a discrete set.

Approximate Universality: A discrete gate set $\mathcal{G}$ generates a dense subgroup in $SU(2^n)$.

Universality – Exact vs. Approximate

Exact Universality: Any unitary $U \in SU(2^n)$ is represented *exactly* by a finite sequence of gates.

- Requires continuous parameter gate sets (e.g., $\{\text{CNOT}, R_z(\theta), R_y(\theta)\}$).
- Uncountably infinite group $SU(2^n)$ cannot be generated by a discrete set.

Approximate Universality: A discrete gate set $\mathcal{G}$ generates a dense subgroup in $SU(2^n)$.

- For any target U and error $\epsilon > 0$, there exists a sequence $G_k \dots G_1$ such that:

$$d(U, G_k \dots G_1) \leq \epsilon$$

Universality – Exact vs. Approximate

Exact Universality: Any unitary $U \in SU(2^n)$ is represented *exactly* by a finite sequence of gates.

- Requires continuous parameter gate sets (e.g., $\{\text{CNOT}, R_z(\theta), R_y(\theta)\}$).
- Uncountably infinite group $SU(2^n)$ cannot be generated by a discrete set.

Approximate Universality: A discrete gate set $\mathcal{G}$ generates a dense subgroup in $SU(2^n)$.

- For any target U and error $\epsilon > 0$, there exists a sequence $G_k \dots G_1$ such that:

$$d(U, G_k \dots G_1) \leq \epsilon$$

- Essential for fault tolerance since only discrete gates can be protected.

Standard Universal Gate Sets

① **Clifford + T Set:** $\{H, S, T, \text{CNOT}\}$

1 Clifford + T Set: $\{H, S, T, \text{CNOT}\}$

- The Clifford group $\{H, S, \text{CNOT}\}$ is classically simulable (Gottesman-Knill).

1 Clifford + T Set: $\{H, S, T, \text{CNOT}\}$

- The Clifford group $\{H, S, \text{CNOT}\}$ is classically simulable (Gottesman-Knill).
- Adding the non-Clifford $T = \text{diag}(1, e^{i\pi/4})$ gate breaks classical simulability and generates a dense subset in $SU(2)$, making it universal.

① Clifford + T Set: $\{H, S, T, \text{CNOT}\}$

- The Clifford group $\{H, S, \text{CNOT}\}$ is classically simulable (Gottesman-Knill).
- Adding the non-Clifford $T = \text{diag}(1, e^{i\pi/4})$ gate breaks classical simulability and generates a dense subset in $SU(2)$, making it universal.
- Standard set for fault-tolerant architectures.

Standard Universal Gate Sets

1 Clifford + T Set: $\{H, S, T, \text{CNOT}\}$

- The Clifford group $\{H, S, \text{CNOT}\}$ is classically simulable (Gottesman-Knill).
- Adding the non-Clifford $T = \text{diag}(1, e^{i\pi/4})$ gate breaks classical simulability and generates a dense subset in $SU(2)$, making it universal.
- Standard set for fault-tolerant architectures.

2 Toffoli + Hadamard Set: $\{\text{Toffoli}, H\}$

Standard Universal Gate Sets

1 Clifford + T Set: $\{H, S, T, \text{CNOT}\}$

- The Clifford group $\{H, S, \text{CNOT}\}$ is classically simulable (Gottesman-Knill).
- Adding the non-Clifford $T = \text{diag}(1, e^{i\pi/4})$ gate breaks classical simulability and generates a dense subset in $SU(2)$, making it universal.
- Standard set for fault-tolerant architectures.

2 Toffoli + Hadamard Set: $\{\text{Toffoli}, H\}$

- The Toffoli (CCNOT) gate is universal for classical reversible circuits.

Standard Universal Gate Sets

1 Clifford + T Set: $\{H, S, T, \text{CNOT}\}$

- The Clifford group $\{H, S, \text{CNOT}\}$ is classically simulable (Gottesman-Knill).
- Adding the non-Clifford $T = \text{diag}(1, e^{i\pi/4})$ gate breaks classical simulability and generates a dense subset in $SU(2)$, making it universal.
- Standard set for fault-tolerant architectures.

2 Toffoli + Hadamard Set: $\{\text{Toffoli}, H\}$

- The Toffoli (CCNOT) gate is universal for classical reversible circuits.
- The Hadamard (H) gate introduces superpositions, yielding full quantum universality.

The Solovay–Kitaev Theorem

Theorem Statement

Any single-qubit unitary $U \in SU(2)$ can be approximated to precision $\epsilon > 0$ by a sequence of gates from a dense set $\mathcal{G}$ (closed under inversion) of length:

$$L = \mathcal{O} \left(\log^c \left(\frac{1}{\epsilon} \right) \right)$$

where $c \approx 3.97$ (and can compile in polynomial-of- $\log(1/\epsilon)$ time).

The Solovay–Kitaev Theorem

Theorem Statement

Any single-qubit unitary $U \in SU(2)$ can be approximated to precision $\epsilon > 0$ by a sequence of gates from a dense set $\mathcal{G}$ (closed under inversion) of length:

$$L = \mathcal{O} \left(\log^c \left(\frac{1}{\epsilon} \right) \right)$$

where $c \approx 3.97$ (and can compile in polynomial-of- $\log(1/\epsilon)$ time).

Significance for FTQC:

- Compiling continuous logic gates into fault-tolerant discrete gates requires only a polylogarithmic overhead.

The Solovay–Kitaev Theorem

Theorem Statement

Any single-qubit unitary $U \in SU(2)$ can be approximated to precision $\epsilon > 0$ by a sequence of gates from a dense set $\mathcal{G}$ (closed under inversion) of length:

$$L = \mathcal{O} \left(\log^c \left(\frac{1}{\epsilon} \right) \right)$$

where $c \approx 3.97$ (and can compile in polynomial-of- $\log(1/\epsilon)$ time).

Significance for FTQC:

- Compiling continuous logic gates into fault-tolerant discrete gates requires only a polylogarithmic overhead.
- Modern synthesis (number-theoretic methods) reduces the scaling to

$$c \approx 1$$

The Stabilizer Formalism

- **Key Idea:** Describe quantum states by tracking the operators that preserve them, rather than tracking state vectors directly.

The Stabilizer Formalism

- **Key Idea:** Describe quantum states by tracking the operators that preserve them, rather than tracking state vectors directly.
- **Pauli Group ($\mathcal{P}_n$):** Tensor products of Pauli matrices $\{I, X, Y, Z\}$ with phases $\{\pm 1, \pm i\}$.

The Stabilizer Formalism

- **Key Idea:** Describe quantum states by tracking the operators that preserve them, rather than tracking state vectors directly.
- **Pauli Group ($\mathcal{P}_n$):** Tensor products of Pauli matrices $\{I, X, Y, Z\}$ with phases $\{\pm 1, \pm i\}$.
- **Stabilizer Group (S):** An abelian subgroup of $\mathcal{P}_n$ not containing $-I$.

The Stabilizer Formalism

- **Key Idea:** Describe quantum states by tracking the operators that preserve them, rather than tracking state vectors directly.
- **Pauli Group ($\mathcal{P}_n$):** Tensor products of Pauli matrices $\{I, X, Y, Z\}$ with phases $\{\pm 1, \pm i\}$.
- **Stabilizer Group (S):** An abelian subgroup of $\mathcal{P}_n$ not containing $-I$.
- **Code Space (V_S):** The common $+1$ eigenspace of all $M \in S$:

$$V_S = \{|\psi\rangle \mid M|\psi\rangle = |\psi\rangle \text{ for all } M \in S\}$$

The Stabilizer Formalism

- **Key Idea:** Describe quantum states by tracking the operators that preserve them, rather than tracking state vectors directly.
- **Pauli Group ($\mathcal{P}_n$):** Tensor products of Pauli matrices $\{I, X, Y, Z\}$ with phases $\{\pm 1, \pm i\}$.
- **Stabilizer Group (S):** An abelian subgroup of $\mathcal{P}_n$ not containing $-I$.
- **Code Space (V_S):** The common $+1$ eigenspace of all $M \in S$:

$$V_S = \{|\psi\rangle \mid M|\psi\rangle = |\psi\rangle \text{ for all } M \in S\}$$

- **Encoding:** $n - k$ independent and commuting generators g_i encode k logical qubits in n physical qubits.

Error Syndrome Extraction

Let $E \in \mathcal{P}_n$ be a Pauli error acting on the logical state $|\psi_L\rangle$. To detect it, we measure the stabilizer generators g_j :

Error Syndrome Extraction

Let $E \in \mathcal{P}_n$ be a Pauli error acting on the logical state $|\psi_L\rangle$. To detect it, we measure the stabilizer generators g_j :

- **Commuting Error** ($[E, g_j] = 0$):

$$g_j(E |\psi_L\rangle) = E g_j |\psi_L\rangle = +1(E |\psi_L\rangle)$$

Syndrome bit is $s_j = 0$ (no error detected by g_j).

- **Anti-commuting Error** ($\{E, g_j\} = 0$):

$$g_j(E |\psi_L\rangle) = -E g_j |\psi_L\rangle = -1(E |\psi_L\rangle)$$

Syndrome bit is $s_j = 1$ (error detected by g_j).

Error Syndrome Extraction

Let $E \in \mathcal{P}_n$ be a Pauli error acting on the logical state $|\psi_L\rangle$. To detect it, we measure the stabilizer generators g_j :

- **Commuting Error** ($[E, g_j] = 0$):

$$g_j(E |\psi_L\rangle) = E g_j |\psi_L\rangle = +1(E |\psi_L\rangle)$$

Syndrome bit is $s_j = 0$ (no error detected by g_j).

- **Anti-commuting Error** ($\{E, g_j\} = 0$):

$$g_j(E |\psi_L\rangle) = -E g_j |\psi_L\rangle = -1(E |\psi_L\rangle)$$

Syndrome bit is $s_j = 1$ (error detected by g_j).

The syndrome string $(s_1, s_2, \dots, s_{n-k})$ identifies the error class without collapsing the encoded superposition.

3-Qubit Bit-Flip & Phase-Flip Codes

Bit-Flip Code (X errors):

- $|0_L\rangle = |000\rangle, |1_L\rangle = |111\rangle$
- Generators: Z_1Z_2, Z_2Z_3
- Syndrome table:
 - $00 \rightarrow$ No Error
 - $10 \rightarrow X_1$ (Q0)
 - $11 \rightarrow X_2$ (Q1)
 - $01 \rightarrow X_3$ (Q2)

3-Qubit Bit-Flip & Phase-Flip Codes

Bit-Flip Code (X errors):

- $|0_L\rangle = |000\rangle$, $|1_L\rangle = |111\rangle$
- Generators: Z_1Z_2 , Z_2Z_3
- Syndrome table:
 - $00 \rightarrow$ No Error
 - $10 \rightarrow X_1$ (Q0)
 - $11 \rightarrow X_2$ (Q1)
 - $01 \rightarrow X_3$ (Q2)

Phase-Flip Code (Z errors):

- $|0_L\rangle = |+++ \rangle$,
 $|1_L\rangle = |-- - \rangle$
- Generators: X_1X_2 , X_2X_3
- Rotated version of bit-flip code into the Hadamard basis.
- Detects Z errors by measuring X_iX_{i+1} stabilizers.

Shor's Nine-Qubit Code

- The smallest code that corrects an arbitrary single-qubit error (X , Z , or $Y = iXZ$).

Shor's Nine-Qubit Code

- The smallest code that corrects an arbitrary single-qubit error (X , Z , or $Y = iXZ$).
- Concatenates the 3-qubit bit-flip and phase-flip codes:

$$|0_L\rangle = \frac{(|000\rangle + |111\rangle)^{\otimes 3}}{2\sqrt{2}}, \quad |1_L\rangle = \frac{(|000\rangle - |111\rangle)^{\otimes 3}}{2\sqrt{2}}$$

Shor's Nine-Qubit Code

- The smallest code that corrects an arbitrary single-qubit error (X , Z , or $Y = iXZ$).
- Concatenates the 3-qubit bit-flip and phase-flip codes:

$$|0_L\rangle = \frac{(|000\rangle + |111\rangle)^{\otimes 3}}{2\sqrt{2}}, \quad |1_L\rangle = \frac{(|000\rangle - |111\rangle)^{\otimes 3}}{2\sqrt{2}}$$

- **Stabilizers (8 generators):**

Shor's Nine-Qubit Code

- The smallest code that corrects an arbitrary single-qubit error (X , Z , or $Y = iXZ$).
- Concatenates the 3-qubit bit-flip and phase-flip codes:

$$|0_L\rangle = \frac{(|000\rangle + |111\rangle)^{\otimes 3}}{2\sqrt{2}}, \quad |1_L\rangle = \frac{(|000\rangle - |111\rangle)^{\otimes 3}}{2\sqrt{2}}$$

- **Stabilizers (8 generators):**

- Bit-flips detected by 6 Z -type stabilizers:

$$Z_1Z_2, Z_2Z_3, \quad Z_4Z_5, Z_5Z_6, \quad Z_7Z_8, Z_8Z_9$$

Shor's Nine-Qubit Code

- The smallest code that corrects an arbitrary single-qubit error (X , Z , or $Y = iXZ$).
- Concatenates the 3-qubit bit-flip and phase-flip codes:

$$|0_L\rangle = \frac{(|000\rangle + |111\rangle)^{\otimes 3}}{2\sqrt{2}}, \quad |1_L\rangle = \frac{(|000\rangle - |111\rangle)^{\otimes 3}}{2\sqrt{2}}$$

- **Stabilizers (8 generators):**

- Bit-flips detected by 6 Z -type stabilizers:

$$Z_1 Z_2, Z_2 Z_3, \quad Z_4 Z_5, Z_5 Z_6, \quad Z_7 Z_8, Z_8 Z_9$$

- Phase-flips detected by 2 X -type stabilizers:

$$X_1 X_2 X_3 X_4 X_5 X_6, \quad X_4 X_5 X_6 X_7 X_8 X_9$$

Topological Codes & Toric Code

- **Topological QEC:** Arranges physical qubits on a 2D/3D lattice. Generators are local, while logical states are stored in global homological properties.

Topological Codes & Toric Code

- **Topological QEC:** Arranges physical qubits on a 2D/3D lattice. Generators are local, while logical states are stored in global homological properties.
- **Kitaev's Toric Code:** Defined on a 2D square lattice wrapped around a torus. Qubits are on the edges.

Topological Codes & Toric Code

- **Topological QEC:** Arranges physical qubits on a 2D/3D lattice. Generators are local, while logical states are stored in global homological properties.
- **Kitaev's Toric Code:** Defined on a 2D square lattice wrapped around a torus. Qubits are on the edges.
- **Local Stabilizers:**

Topological Codes & Toric Code

- **Topological QEC:** Arranges physical qubits on a 2D/3D lattice. Generators are local, while logical states are stored in global homological properties.
- **Kitaev's Toric Code:** Defined on a 2D square lattice wrapped around a torus. Qubits are on the edges.
- **Local Stabilizers:**
 - **Star (Vertex) Operator:** $A_s = \prod_{j \in \text{star}(s)} X_j$

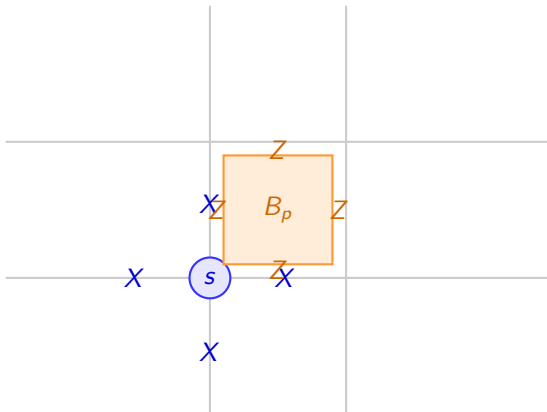
Topological Codes & Toric Code

- **Topological QEC:** Arranges physical qubits on a 2D/3D lattice. Generators are local, while logical states are stored in global homological properties.
- **Kitaev's Toric Code:** Defined on a 2D square lattice wrapped around a torus. Qubits are on the edges.
- **Local Stabilizers:**
 - **Star (Vertex) Operator:** $A_s = \prod_{j \in \text{star}(s)} X_j$
 - **Plaquette (Face) Operator:** $B_p = \prod_{j \in \partial p} Z_j$

Topological Codes & Toric Code

- **Topological QEC:** Arranges physical qubits on a 2D/3D lattice. Generators are local, while logical states are stored in global homological properties.
- **Kitaev's Toric Code:** Defined on a 2D square lattice wrapped around a torus. Qubits are on the edges.
- **Local Stabilizers:**
 - **Star (Vertex) Operator:** $A_s = \prod_{j \in \text{star}(s)} X_j$
 - **Plaquette (Face) Operator:** $B_p = \prod_{j \in \partial p} Z_j$
- **Commutation:** A_s and B_p commute since star and plaquette boundaries share either 0 or 2 edges, cancelling out phase factors $((-1)^2 = 1)$.

Toric Code Local Operators



- **Anyonic Excitations:**

- **Anyonic Excitations:**

- Z-error chains create electric anyons (e) at endpoints (violations of $A_s = -1$).

- **Anyonic Excitations:**

- Z -error chains create electric anyons (e) at endpoints (violations of $A_s = -1$).
- X -error chains create magnetic anyons (m) at endpoints (violations of $B_p = -1$).

- **Anyonic Excitations:**

- Z -error chains create electric anyons (e) at endpoints (violations of $A_s = -1$).
- X -error chains create magnetic anyons (m) at endpoints (violations of $B_p = -1$).

- **Decoders:** Syndromes measure anyon endpoints. Algorithms (e.g. MWPM) pair up anyons to annihilate them.

- **Anyonic Excitations:**

- Z -error chains create electric anyons (e) at endpoints (violations of $A_s = -1$).
- X -error chains create magnetic anyons (m) at endpoints (violations of $B_p = -1$).

- **Decoders:** Syndromes measure anyon endpoints. Algorithms (e.g. MWPM) pair up anyons to annihilate them.

- **Logical Errors:** Require error chains that wrap completely around the torus. Probability decreases exponentially with lattice size L (distance $d = L$).

- **Anyonic Excitations:**

- Z -error chains create electric anyons (e) at endpoints (violations of $A_s = -1$).
- X -error chains create magnetic anyons (m) at endpoints (violations of $B_p = -1$).

- **Decoders:** Syndromes measure anyon endpoints. Algorithms (e.g. MWPM) pair up anyons to annihilate them.

- **Logical Errors:** Require error chains that wrap completely around the torus. Probability decreases exponentially with lattice size L (distance $d = L$).

- **Surface Code:** Flat 2D layout with rough/smooth boundaries. Highly practical, leading candidate for hardware (threshold $\sim 1\%$).

Fault-Tolerant Gates & Eastin-Knill Theorem

- **Transversal Gates:** Gates applied bitwise to physical qubits in the code block. Naturally fault-tolerant (errors do not spread).

Fault-Tolerant Gates & Eastin-Knill Theorem

- **Transversal Gates:** Gates applied bitwise to physical qubits in the code block. Naturally fault-tolerant (errors do not spread).

Fault-Tolerant Gates & Eastin-Knill Theorem

- **Transversal Gates:** Gates applied bitwise to physical qubits in the code block. Naturally fault-tolerant (errors do not spread).

The Eastin-Knill Theorem

No quantum error-correcting code can implement a universal set of logical gates transversally.

Fault-Tolerant Gates & Eastin-Knill Theorem

- **Transversal Gates:** Gates applied bitwise to physical qubits in the code block. Naturally fault-tolerant (errors do not spread).

The Eastin-Knill Theorem

No quantum error-correcting code can implement a universal set of logical gates transversally.

Implication:

- Stabilizer codes implement Clifford gates transversally.

Fault-Tolerant Gates & Eastin-Knill Theorem

- **Transversal Gates:** Gates applied bitwise to physical qubits in the code block. Naturally fault-tolerant (errors do not spread).

The Eastin-Knill Theorem

No quantum error-correcting code can implement a universal set of logical gates transversally.

Implication:

- Stabilizer codes implement Clifford gates transversally.
- The non-Clifford T gate must be implemented via **Magic State Distillation and Injection**, which is resource-intensive.

Threshold Theorem & Resource Scaling

The Threshold Theorem

If the physical error rate per gate p is below a critical threshold p_{th} , the logical error rate ϵ_L can be made arbitrarily small by increasing the code distance d :

$$\epsilon_L \approx \mathcal{O} \left(\left(\frac{p}{p_{\text{th}}} \right)^{\frac{d+1}{2}} \right)$$

Threshold Theorem & Resource Scaling

The Threshold Theorem

If the physical error rate per gate p is below a critical threshold p_{th} , the logical error rate ϵ_L can be made arbitrarily small by increasing the code distance d :

$$\epsilon_L \approx \mathcal{O} \left(\left(\frac{p}{p_{\text{th}}} \right)^{\frac{d+1}{2}} \right)$$

Resource Overhead Scaling:

- The physical qubit count N needed to achieve a target logical error rate ϵ_L scales as:

$$N \sim \mathcal{O} \left(\log^2 \left(\frac{1}{\epsilon_L} \right) \right)$$

Threshold Theorem & Resource Scaling

The Threshold Theorem

If the physical error rate per gate p is below a critical threshold p_{th} , the logical error rate ϵ_L can be made arbitrarily small by increasing the code distance d :

$$\epsilon_L \approx \mathcal{O} \left(\left(\frac{p}{p_{\text{th}}} \right)^{\frac{d+1}{2}} \right)$$

Resource Overhead Scaling:

- The physical qubit count N needed to achieve a target logical error rate ϵ_L scales as:

$$N \sim \mathcal{O} \left(\log^2 \left(\frac{1}{\epsilon_L} \right) \right)$$

- This polylogarithmic overhead guarantees the physical viability of scalable quantum computers.

Problem 1: Stabilizer Commutation

- Logical states $|0_L\rangle = |000\rangle$ and $|1_L\rangle = |111\rangle$ are $+1$ eigenstates of Z_1Z_2 and Z_2Z_3 .
- A bit-flip error X_1 on qubit 0 anti-commutes with Z_1Z_2 :

$$(Z_1Z_2)X_1|\psi_L\rangle = -X_1(Z_1Z_2)|\psi_L\rangle = -1(X_1|\psi_L\rangle)$$

yielding a -1 syndrome, indicating an error.

Problem 1: Stabilizer Commutation

- Logical states $|0_L\rangle = |000\rangle$ and $|1_L\rangle = |111\rangle$ are $+1$ eigenstates of Z_1Z_2 and Z_2Z_3 .
- A bit-flip error X_1 on qubit 0 anti-commutes with Z_1Z_2 :

$$(Z_1Z_2)X_1|\psi_L\rangle = -X_1(Z_1Z_2)|\psi_L\rangle = -1(X_1|\psi_L\rangle)$$

yielding a -1 syndrome, indicating an error.

Problem 2: Eastin-Knill Implications

- Because no code can implement a universal gate set transversally, fault-tolerant computers require a hybrid approach: transversal Clifford gates + a "magic state factory" to distill and inject T gates.

Problem 3: Shor Code Correction of Y_1 Error

- A Pauli Y error on qubit 0 is $Y_1 = iX_1Z_1$ (combined bit-flip and phase-flip).

Problem 3: Shor Code Correction of Y_1 Error

- A Pauli Y error on qubit 0 is $Y_1 = iX_1Z_1$ (combined bit-flip and phase-flip).
- **Bit-flip Detection:** Y_1 anti-commutes with stabilizer Z_1Z_2 , signaling a bit-flip on qubit 0 (since it commutes with Z_2Z_3).

Problem 3: Shor Code Correction of Y_1 Error

- A Pauli Y error on qubit 0 is $Y_1 = iX_1Z_1$ (combined bit-flip and phase-flip).
- **Bit-flip Detection:** Y_1 anti-commutes with stabilizer Z_1Z_2 , signaling a bit-flip on qubit 0 (since it commutes with Z_2Z_3).
- **Phase-flip Detection:** Y_1 anti-commutes with stabilizer $g_x = X_1X_2X_3X_4X_5X_6$, signaling a phase-flip on the first block of three qubits.

Problem 3: Shor Code Correction of Y_1 Error

- A Pauli Y error on qubit 0 is $Y_1 = iX_1Z_1$ (combined bit-flip and phase-flip).
- **Bit-flip Detection:** Y_1 anti-commutes with stabilizer Z_1Z_2 , signaling a bit-flip on qubit 0 (since it commutes with Z_2Z_3).
- **Phase-flip Detection:** Y_1 anti-commutes with stabilizer $g_x = X_1X_2X_3X_4X_5X_6$, signaling a phase-flip on the first block of three qubits.
- **Correction:** Applying X_1 and Z_1 (which is Y_1 up to global phase) completely corrects the state.

Take-away Messages

- 1 QEC uses stabilizer measurements to extract error syndromes without destroying logical superposition states.

Take-away Messages

- 1 QEC uses stabilizer measurements to extract error syndromes without destroying logical superposition states.
- 2 Topological codes (like surface codes) utilize local stabilizers and global topology, offering high error thresholds ($\sim 1\%$) and practical 2D routing.

Take-away Messages

- 1 QEC uses stabilizer measurements to extract error syndromes without destroying logical superposition states.
- 2 Topological codes (like surface codes) utilize local stabilizers and global topology, offering high error thresholds ($\sim 1\%$) and practical 2D routing.
- 3 The Eastin-Knill theorem prohibits fully transversal universal codes, requiring magic state distillation for non-Clifford gates.

Take-away Messages

- 1 QEC uses stabilizer measurements to extract error syndromes without destroying logical superposition states.
- 2 Topological codes (like surface codes) utilize local stabilizers and global topology, offering high error thresholds ($\sim 1\%$) and practical 2D routing.
- 3 The Eastin-Knill theorem prohibits fully transversal universal codes, requiring magic state distillation for non-Clifford gates.
- 4 The threshold theorem guarantees that large-scale fault-tolerant quantum computation is feasible with polylogarithmic physical qubit overhead.